

2026.03.04.수

🕒 생성일	@2026년 3월 4일 오전 9:08
🏷 태그	

목차

1. Terraform
- 2.

▼ 1) Terraform

- 설치
<https://developer.hashicorp.com/terraform>
- tfenv
 - terraform 을 가상 환경으로 구성하는 방법으로 여러 버전으로 설치하고 관리할 때 유용
 - 전역으로 관리되기에 선택된 버전이 쉘 전체에 적용, 격리 환경 아님
- tfenv 설치

```
# 깃 클론
git clone https://github.com/tfutils/tfenv.git ~/.tfenv

# 환경 변수 등록
echo 'export PATH="$HOME/.tfenv/bin:$PATH"' >> ~/.bash_profile

# 환경 변수 확인
source ~/.bash_profile
```

- open tofu
 - terraform과 똑같은 역할을 하는 오픈소스
 - tenv
- 실습: key-pair 생성하고 terraform 등록, terraform이 aws에 key-pair 등록
 - 키 생성 (private, public 2개 동시에 생성됨)

```
ssh-keygen -t rsa -b 2048 -C "" -f "$HOME/.ssh/<키페어 이름>" -N "" && sudo cp /<경로>/<키페어 이름> /<경로>/<키페어 이름>.pem
```

- ~/terraform/0.key/key.tf 파일 작성

```
provider "aws" {
  region = "ap-northeast-2"
}

resource "aws_key_pair" "ec2_key" {
  key_name   = "tf-ec2-key"
  public_key = file("~/ssh/tf-ec2-key.pub")
}
```

- terraform 실행부터 삭제까지

```
# 초기화
terraform init

# 현재 상태를 보고 예상되는 리소스 변화 출력
```

```

terraform plan

# 실제 인프라에 반영
terraform apply

# 현재 관리중인 리소스 삭제
terraform destroy

```

▼ ami-id 갱신하는 ec2 생성 terraform 실습

```

provider "aws" {
  region = "ap-northeast-2"
}

```

```

/* 주석 처리
provider "aws" {
  region = "ap-northeast-2"
}
*/

data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    # values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
    values = ["ubuntu/images/hvm-ssd/gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  owners = ["099720109477"] # Canonical
}

resource "aws_instance" "tf-amz-ubuntu" {
#  ami           = "ami-0389ea382ca31bd7f"
  ami = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  tags = {
    Name = "sg0304-ubuntu"
  }
}

resource "aws_instance" "tf-amz-linux" {
  ami           = "resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
  instance_type = "t3.micro"

  tags = {
    Name = "sg0304-amazon-linux"
  }
}

```